
DENNIS GUNAWAN



UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

IF351 DATABASE SYSTEMS

06 SELECT STATEMENT 1



REVIEW:

DATA DEFINITION LANGUAGE (DDL) AND DATA MANIPULATION LANGUAGE (DML) STATEMENTS

- Creating, Showing, & Deleting D atabases
- Selecting the Current D atabase
- Data Types of Columns
- Creating, Showing, & Deleting Tables
- Populating Tables with Data
- Querying Tables
- Updating & Deleting Rows
- Copying Tables
- Populating a Table with Rows from Another Table

COURSE SUB LEARNING OUTCOMES (SUB-CLO)

- Sub-CLO 7
 - Students are able to use SELECT statement (C3)

OUTLINE

- SELECT
- FROM
- WHERE
- Operators
- Aggregation Functions

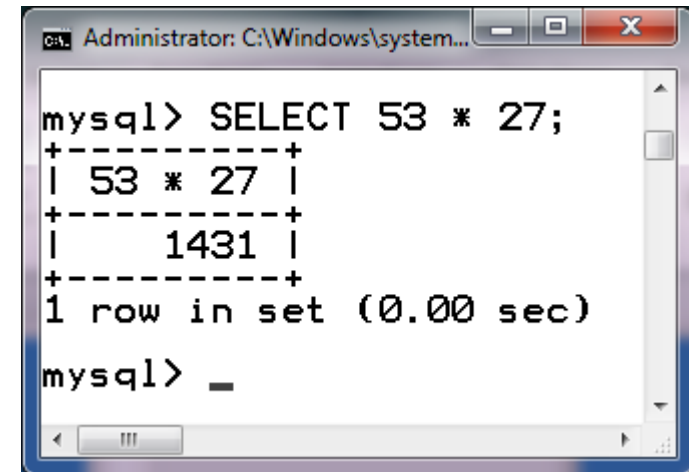
STRUCTURE OF SELECT STATEMENT

```
SELEC  
T      . . .  
FROM   . . .  
WHERE  . . .  
GROUP BY . . .  
HAVIN  
G      . . .  
ORDER BY . . .  
LIMIT  . . . ;
```

SMALLEST SELECT STATEMENT

- The smallest SELECT statement that can be specified consists of one select block with just a SELECT clause
- If no FROM clause is specified, the statement returns a result consisting of one row

```
SELECT 53 * 27;
```



```
Administrator: C:\Windows\system...
mysql> SELECT 53 * 27;
+-----+
| 53 * 27 |
+-----+
|      1431      |
+-----+
1 row in set (0.00 sec)

mysql> _
```

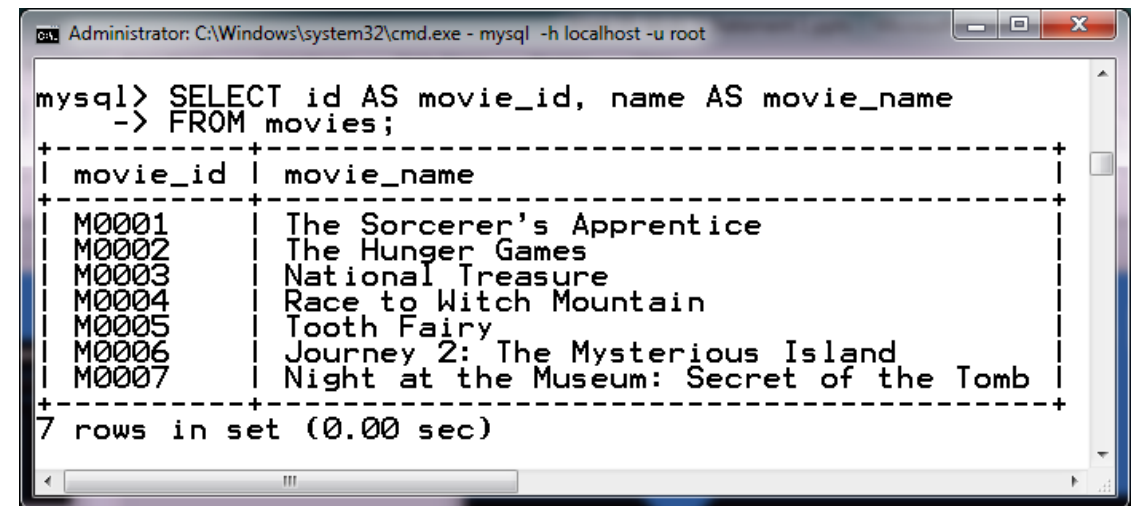
ASSIGNING NAMES TO RESULT COLUMNS

- Specifying an alternative name after an expression in a SELECT clause assigns a name to the matching result column

```
SELECT column_name AS alternative_column_name, ...  
FROM table_name;
```

ASSIGNING NAMES TO RESULT COLUMNS

```
SELECT id AS movie_id, name AS movie_name  
FROM movies;
```



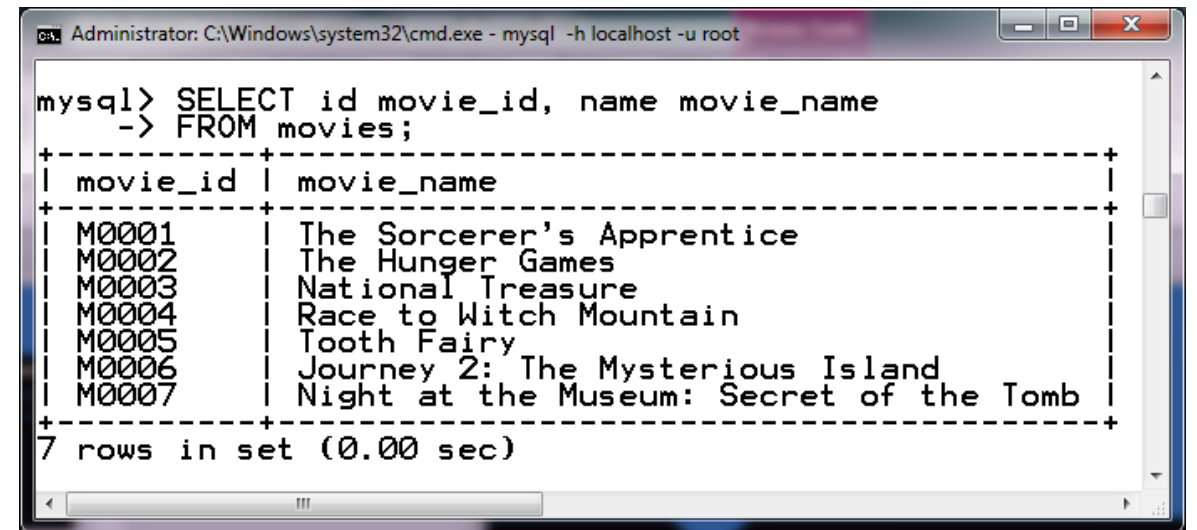
A screenshot of a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". The prompt shows the execution of a MySQL query: `mysql> SELECT id AS movie_id, name AS movie_name -> FROM movies;`. The output is a table with two columns: `movie_id` and `movie_name`. The table contains seven rows of data, each representing a movie. The output is formatted with dashed lines around the table structure.

movie_id	movie_name
M0001	The Sorcerer's Apprentice
M0002	The Hunger Games
M0003	National Treasure
M0004	Race to Witch Mountain
M0005	Tooth Fairy
M0006	Journey 2: The Mysterious Island
M0007	Night at the Museum: Secret of the Tomb

7 rows in set (0.00 sec)

ASSIGNING NAMES TO RESULT COLUMNS

```
SELECT id movie_id, name movie_name  
FROM movies;
```

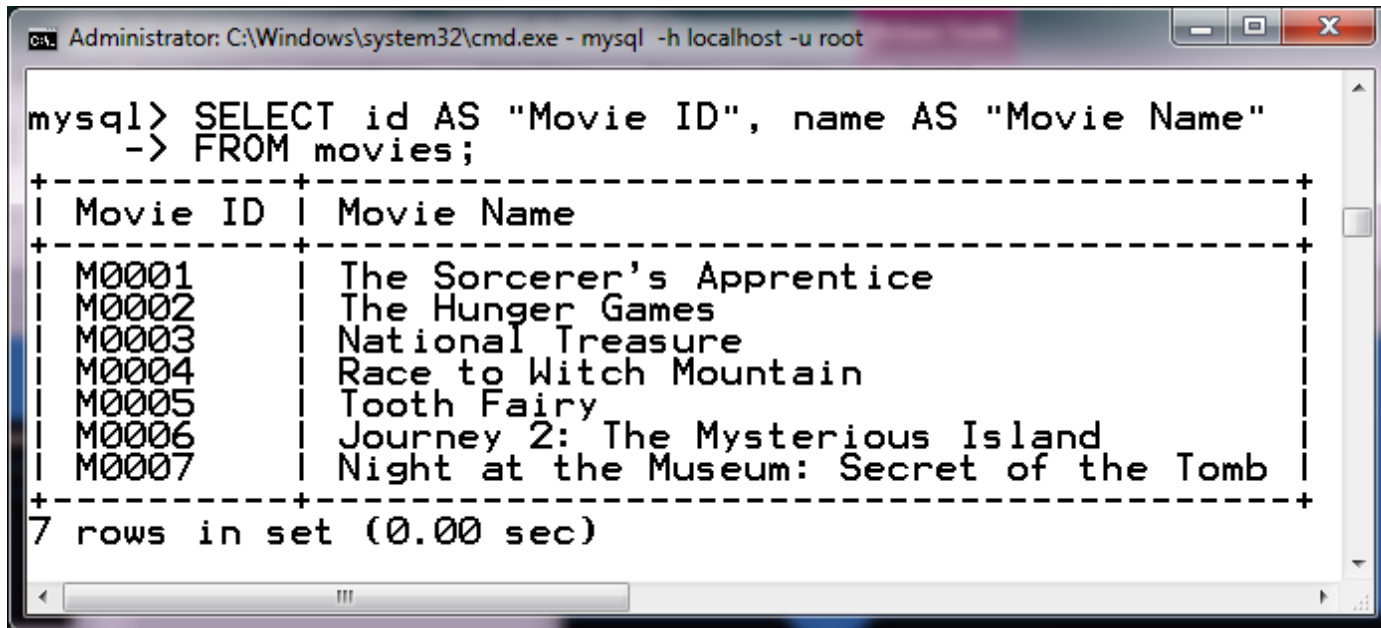


A screenshot of a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". The prompt shows a MySQL session where the user has entered the command: `mysql> SELECT id movie_id, name movie_name -> FROM movies;`. The output is a table with two columns: `movie_id` and `movie_name`. The table contains seven rows of data. Below the table, it says "7 rows in set (0.00 sec)".

movie_id	movie_name
M0001	The Sorcerer's Apprentice
M0002	The Hunger Games
M0003	National Treasure
M0004	Race to Witch Mountain
M0005	Tooth Fairy
M0006	Journey 2: The Mysterious Island
M0007	Night at the Museum: Secret of the Tomb

ASSIGNING NAMES TO RESULT COLUMNS

```
SELECT id AS "Movie ID", name AS "Movie Name"  
FROM movies;
```



The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". The prompt is "mysql>". The user has entered the query: "SELECT id AS 'Movie ID', name AS 'Movie Name' -> FROM movies;". The output is a table with two columns: "Movie ID" and "Movie Name". The table contains 7 rows of data. The output is formatted with a dashed border. Below the table, it says "7 rows in set (0.00 sec)".

```
mysql> SELECT id AS "Movie ID", name AS "Movie Name"  
-> FROM movies;  
+-----+-----+  
| Movie ID | Movie Name  
+-----+-----+  
| M0001   | The Sorcerer's Apprentice  
| M0002   | The Hunger Games  
| M0003   | National Treasure  
| M0004   | Race to Witch Mountain  
| M0005   | Tooth Fairy  
| M0006   | Journey 2: The Mysterious Island  
| M0007   | Night at the Museum: Secret of the Tomb  
+-----+-----+  
7 rows in set (0.00 sec)
```

ASSIGNING NAMES TO RESULT COLUMNS

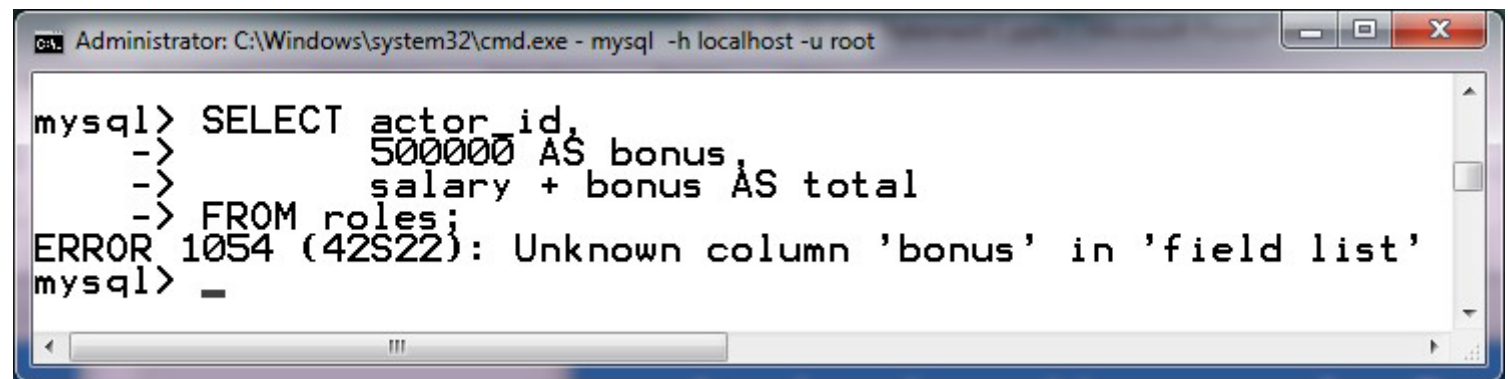
```
SELECT 98 * 97 AS Multiplication Example;
```

```
SELECT 98 * 97 "Multiplication Example";
```

ASSIGNING NAMES TO RESULT COLUMNS

- New column names cannot be used in the same SELECT clause

```
SELECT actor_id,  
       500000 AS bonus,  
       salary + bonus AS total  
FROM roles;
```



A screenshot of a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". The window shows a MySQL session where the user enters a SELECT query. The query is: `mysql> SELECT actor_id, 500000 AS bonus, salary + bonus AS total FROM roles;`. The prompt shows the query being entered line by line. After the query is executed, an error message is displayed: `ERROR 1054 (42S22): Unknown column 'bonus' in 'field list'`. The prompt then shows `mysql> _`.

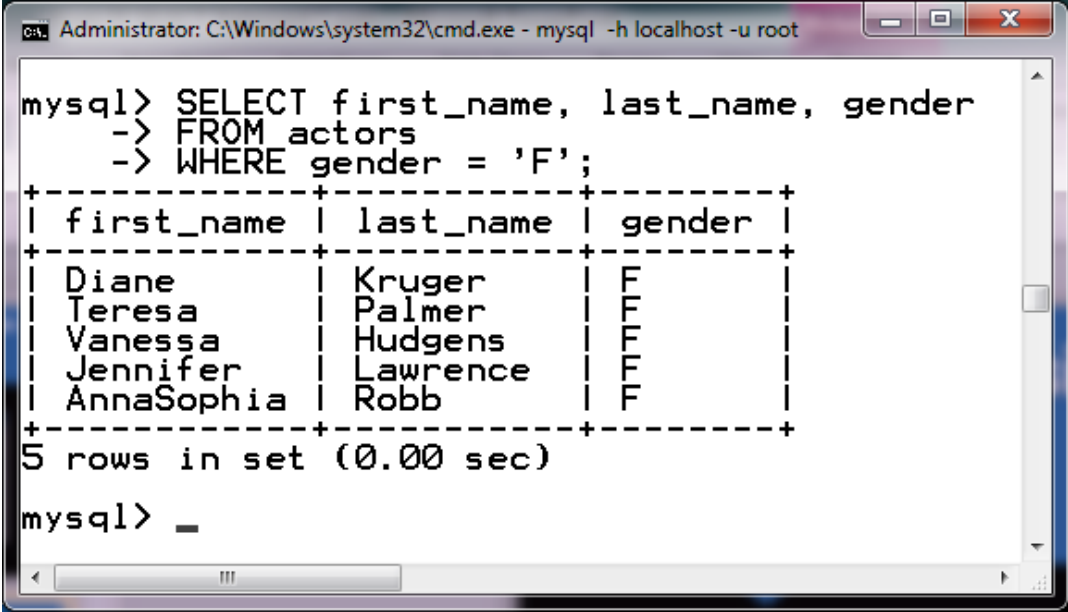
```
Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root  
mysql> SELECT actor_id,  
->          500000 AS bonus,  
->          salary + bonus AS total  
-> FROM roles;  
ERROR 1054 (42S22): Unknown column 'bonus' in 'field list'  
mysql> _
```

CONDITIONS USING COMPARISON OPERATORS

Comparison Operator	Meaning
=	Equal to
<=>	Equal to also if null
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
<>	Not equal to
!=	Not equal to

CONDITIONS USING COMPARISON OPERATORS

```
SELECT first_name, last_name, gender  
FROM actors  
WHERE gender = 'F';
```



A screenshot of a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". The window shows a MySQL session with the following commands and output:

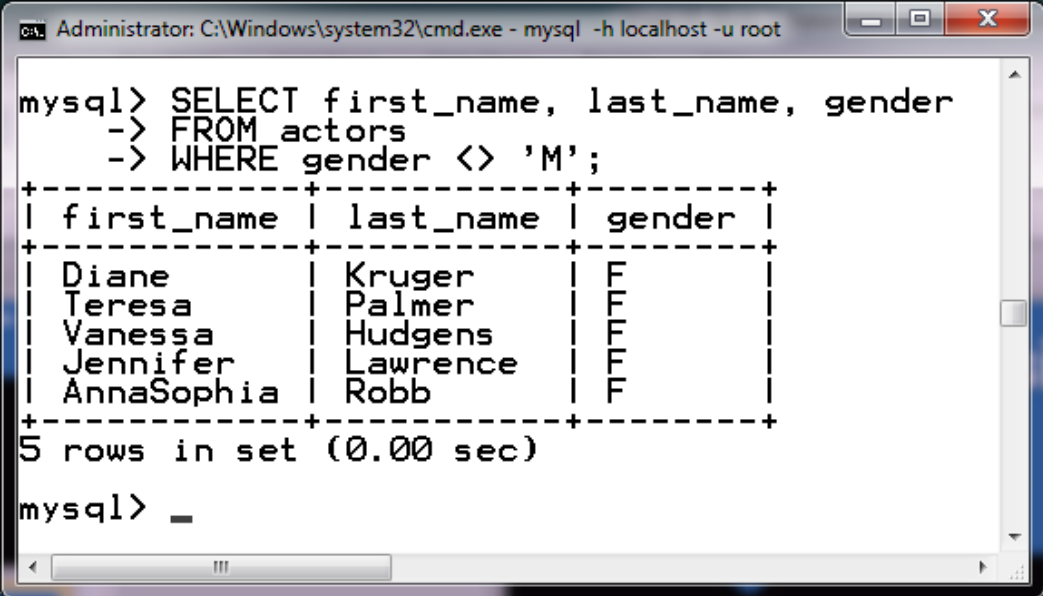
```
mysql> SELECT first_name, last_name, gender  
-> FROM actors  
-> WHERE gender = 'F';
```

first_name	last_name	gender
Diane	Kruger	F
Teresa	Palmer	F
Vanessa	Hudgens	F
Jennifer	Lawrence	F
AnnaSophia	Robb	F

```
5 rows in set (0.00 sec)  
  
mysql> _
```

CONDITIONS USING COMPARISON OPERATORS

```
SELECT first_name, last_name, gender  
FROM actors  
WHERE gender <> 'M';
```



The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". The prompt is "mysql>". The user has entered the following SQL query:

```
mysql> SELECT first_name, last_name, gender  
-> FROM actors  
-> WHERE gender <> 'M';
```

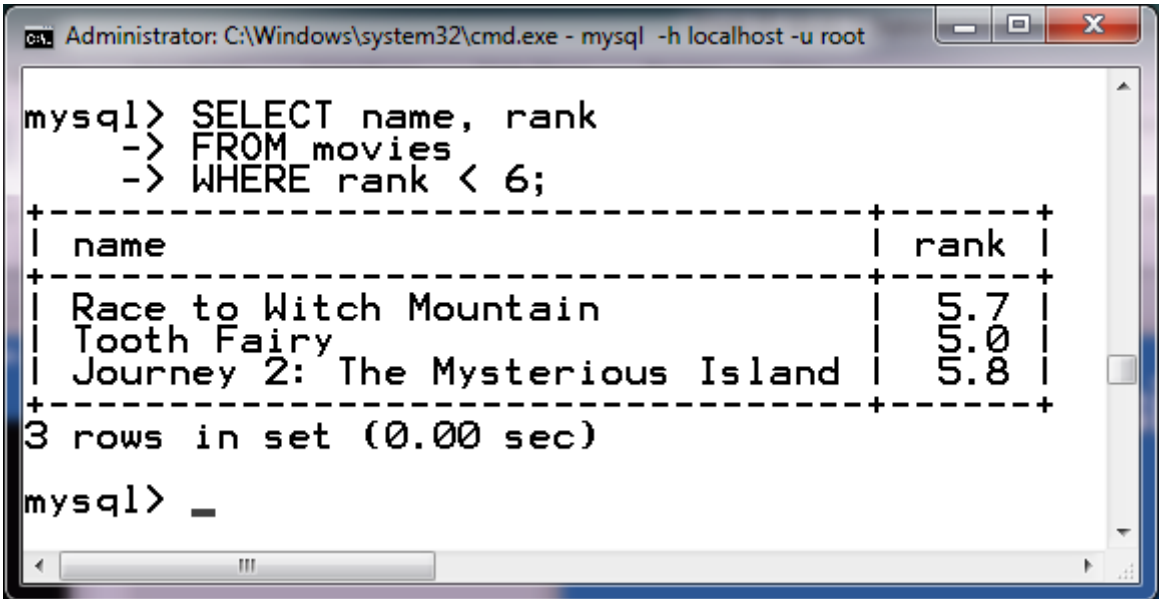
The query results are displayed in a table format with dashed lines for headers and footers. The table has three columns: first_name, last_name, and gender. There are five rows of data, all with gender 'F'.

first_name	last_name	gender
Diane	Kruger	F
Teresa	Palmer	F
Vanessa	Hudgens	F
Jennifer	Lawrence	F
AnnaSophia	Robb	F

Below the table, the output shows "5 rows in set (0.00 sec)". The prompt is now "mysql> _".

CONDITIONS USING COMPARISON OPERATORS

```
SELECT name, rank  
FROM movies  
WHERE rank < 6;
```



A screenshot of a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". The prompt shows a MySQL query being executed: `mysql> SELECT name, rank`, `-> FROM movies`, `-> WHERE rank < 6;`. The output is a table with two columns: "name" and "rank". The table contains three rows of data: "Race to Witch Mountain" with a rank of 5.7, "Tooth Fairy" with a rank of 5.0, and "Journey 2: The Mysterious Island" with a rank of 5.8. Below the table, it says "3 rows in set (0.00 sec)". The prompt ends with `mysql> _`.

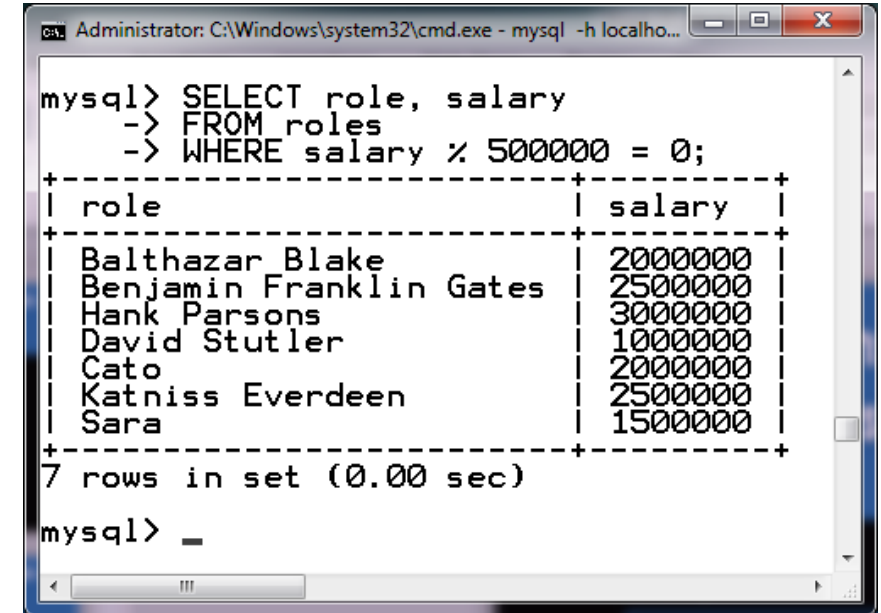
```
mysql> SELECT name, rank  
-> FROM movies  
-> WHERE rank < 6;  
+-----+-----+  
| name                                | rank |  
+-----+-----+  
| Race to Witch Mountain              | 5.7   |  
| Tooth Fairy                         | 5.0   |  
| Journey 2: The Mysterious Island     | 5.8   |  
+-----+-----+  
3 rows in set (0.00 sec)  
mysql> _
```

CONDITIONS USING COMPARISON OPERATORS

SELECT role, salary

FROM roles

WHERE salary % 500000 = 0;



A screenshot of a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localho...". The window shows a MySQL session with the following commands and output:

```
mysql> SELECT role, salary
-> FROM roles
-> WHERE salary % 500000 = 0;
```

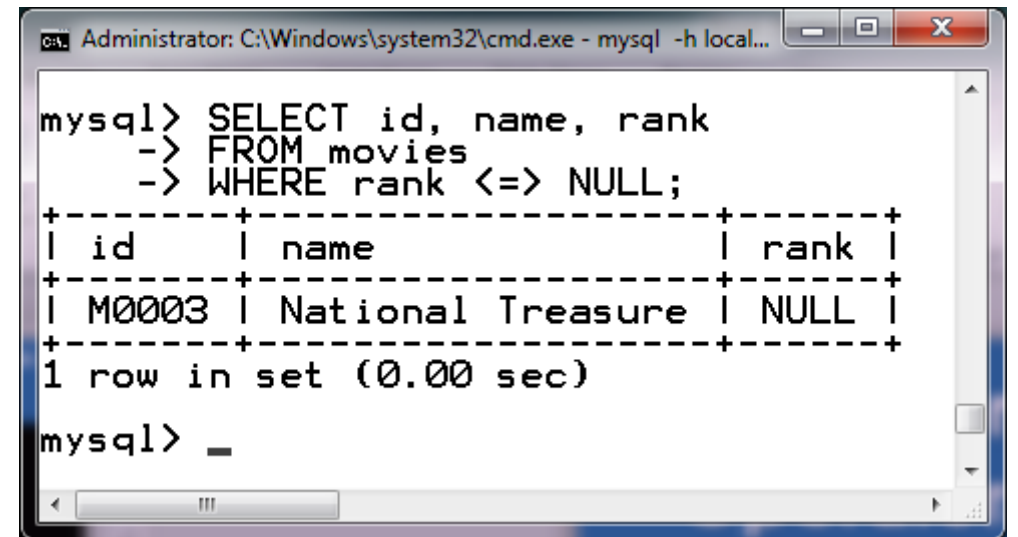
role	salary
Balthazar Blake	2000000
Benjamin Franklin Gates	2500000
Hank Parsons	3000000
David Stutler	1000000
Cato	2000000
Katniss Everdeen	2500000
Sara	1500000

7 rows in set (0.00 sec)

```
mysql> _
```

CONDITIONS USING COMPARISON OPERATORS

```
SELECT id, name, rank  
FROM movies  
WHERE rank <=> NULL;
```



The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h local...". Inside the window, a MySQL query is executed: `mysql> SELECT id, name, rank
-> FROM movies
-> WHERE rank <=> NULL;`. The output is a table with three columns: `id`, `name`, and `rank`. The table contains one row: `M0003`, `National Treasure`, and `NULL`. Below the table, it says "1 row in set (0.00 sec)". The prompt `mysql> _` is shown at the bottom.

```
mysql> SELECT id, name, rank  
-> FROM movies  
-> WHERE rank <=> NULL;  
+-----+-----+-----+  
| id    | name                | rank |  
+-----+-----+-----+  
| M0003 | National Treasure   | NULL |  
+-----+-----+-----+  
1 row in set (0.00 sec)  
mysql> _
```

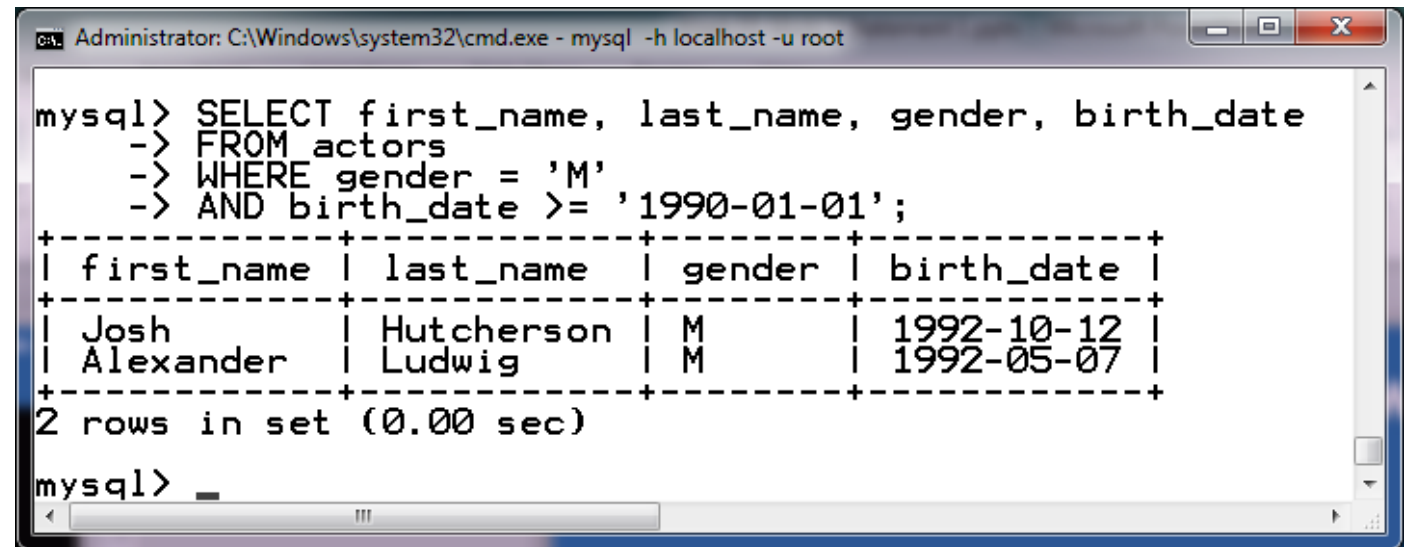
CONDITIONS COUPLED WITH AND, OR, XOR, AND NOT

SELECT first_name, last_name, gender, birth_date

FROM actors

WHERE gender = 'M'

AND birth_date >= '1990-01-01';



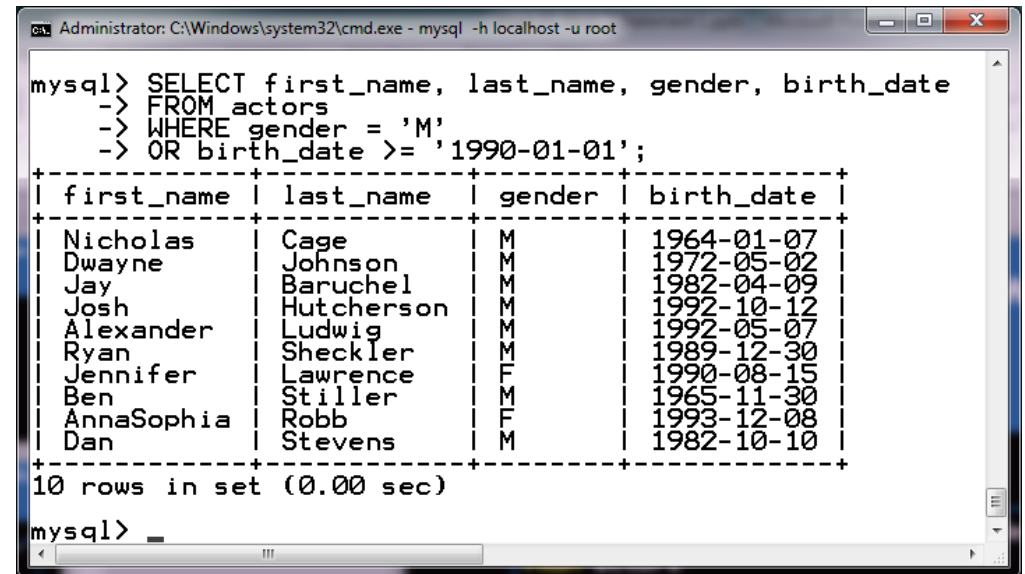
```
C:\Windows\system32\cmd.exe - mysql -h localhost -u root

mysql> SELECT first_name, last_name, gender, birth_date
      -> FROM actors
      -> WHERE gender = 'M'
      -> AND birth_date >= '1990-01-01';
+-----+-----+-----+-----+
| first_name | last_name | gender | birth_date |
+-----+-----+-----+-----+
| Josh       | Hutcherson | M      | 1992-10-12 |
| Alexander  | Ludwig     | M      | 1992-05-07 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> _
```

CONDITIONS COUPLED WITH AND, OR, XOR, AND NOT

```
SELECT first_name, last_name, gender, birth_date
FROM actors
WHERE gender = 'M'
OR birth_date >= '1990-01-01';
```



The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". The prompt is "mysql>". The user has entered the following SQL query:

```
mysql> SELECT first_name, last_name, gender, birth_date
-> FROM actors
-> WHERE gender = 'M'
-> OR birth_date >= '1990-01-01';
```

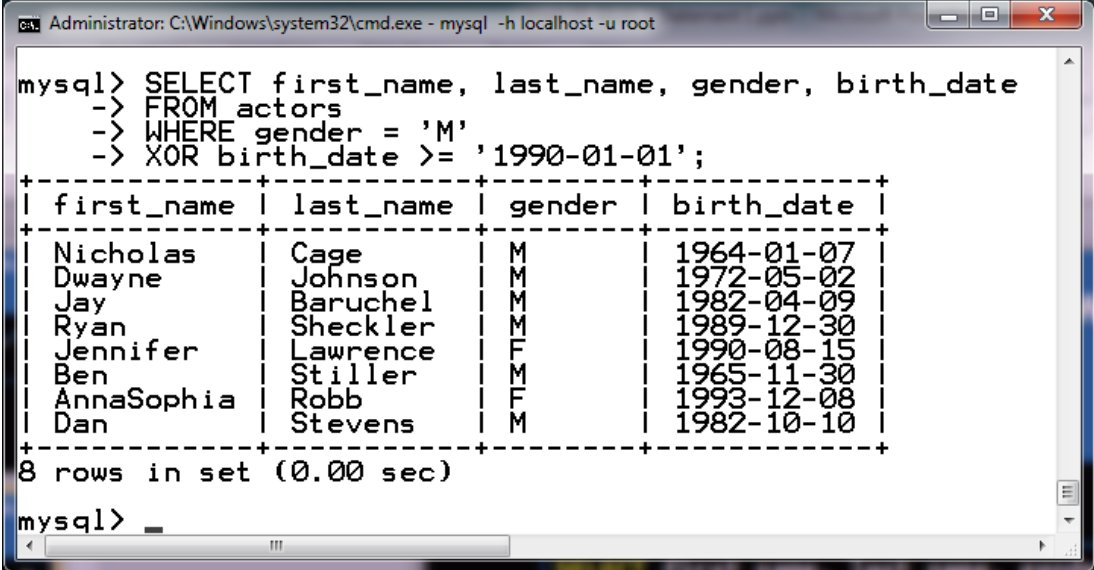
The query results are displayed in a table format with dashed lines as delimiters. The table has four columns: first_name, last_name, gender, and birth_date. There are 10 rows of data.

first_name	last_name	gender	birth_date
Nicholas	Cage	M	1964-01-07
Dwayne	Johnson	M	1972-05-02
Jay	Baruchel	M	1982-04-09
Josh	Hutcherson	M	1992-10-12
Alexander	Ludwig	M	1992-05-07
Ryan	Sheckler	M	1989-12-30
Jennifer	Lawrence	F	1990-08-15
Ben	Stiller	M	1965-11-30
AnnaSophia	Robb	F	1993-12-08
Dan	Stevens	M	1982-10-10

Below the table, the output "10 rows in set (0.00 sec)" is shown. The prompt "mysql> _" is visible at the bottom.

CONDITIONS COUPLED WITH AND, OR, XOR, AND NOT

```
SELECT first_name, last_name, gender, birth_date
FROM actors
WHERE gender = 'M'
XOR birth_date >= '1990-01-01';
```



The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". The prompt is "mysql>". The user has entered the following query:

```
mysql> SELECT first_name, last_name, gender, birth_date
-> FROM actors
-> WHERE gender = 'M'
-> XOR birth_date >= '1990-01-01';
```

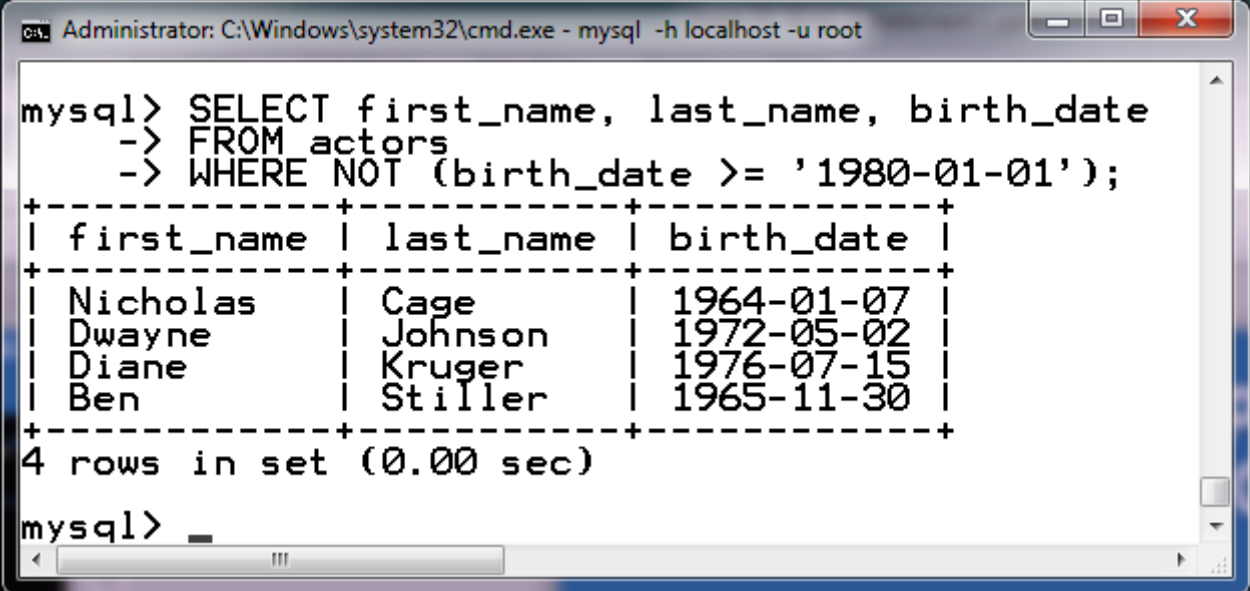
The query returns 8 rows of data, displayed in a table format with dashed lines as delimiters. The columns are first_name, last_name, gender, and birth_date. The data is as follows:

first_name	last_name	gender	birth_date
Nicholas	Cage	M	1964-01-07
Dwayne	Johnson	M	1972-05-02
Jay	Baruchel	M	1982-04-09
Ryan	Sheckler	M	1989-12-30
Jennifer	Lawrence	F	1990-08-15
Ben	Stiller	M	1965-11-30
AnnaSophia	Robb	F	1993-12-08
Dan	Stevens	M	1982-10-10

Below the table, the output indicates "8 rows in set (0.00 sec)". The prompt "mysql> _" is visible at the bottom.

CONDITIONS COUPLED WITH AND, OR, XOR, AND NOT

```
SELECT first_name, last_name, birth_date  
FROM actors  
WHERE NOT (birth_date >= '1980-01-01');
```



The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". The prompt is "mysql>". The user has entered the following SQL query:

```
mysql> SELECT first_name, last_name, birth_date  
-> FROM actors  
-> WHERE NOT (birth_date >= '1980-01-01');
```

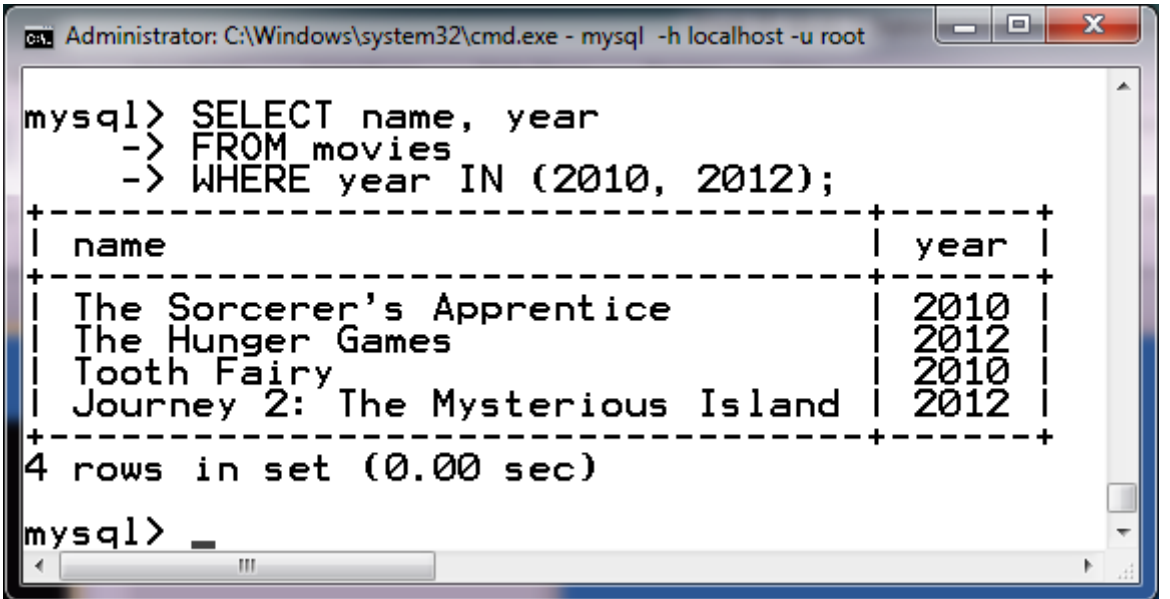
The query returns a table with 4 rows and 3 columns: first_name, last_name, and birth_date. The results are displayed in a table format with dashed lines for column boundaries.

first_name	last_name	birth_date
Nicholas	Cage	1964-01-07
Dwayne	Johnson	1972-05-02
Diane	Kruger	1976-07-15
Ben	Stiller	1965-11-30

Below the table, the output indicates "4 rows in set (0.00 sec)". The prompt "mysql>" is shown again at the bottom.

IN OPERATOR

```
SELECT name, year  
FROM movies  
WHERE year IN (2010, 2012);
```



The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". The prompt shows the execution of a MySQL query: `mysql> SELECT name, year
-> FROM movies
-> WHERE year IN (2010, 2012);`. The output is a table with two columns: "name" and "year". The table contains four rows of data: "The Sorcerer's Apprentice" (2010), "The Hunger Games" (2012), "Tooth Fairy" (2010), and "Journey 2: The Mysterious Island" (2012). Below the table, it says "4 rows in set (0.00 sec)". The prompt ends with `mysql> _`.

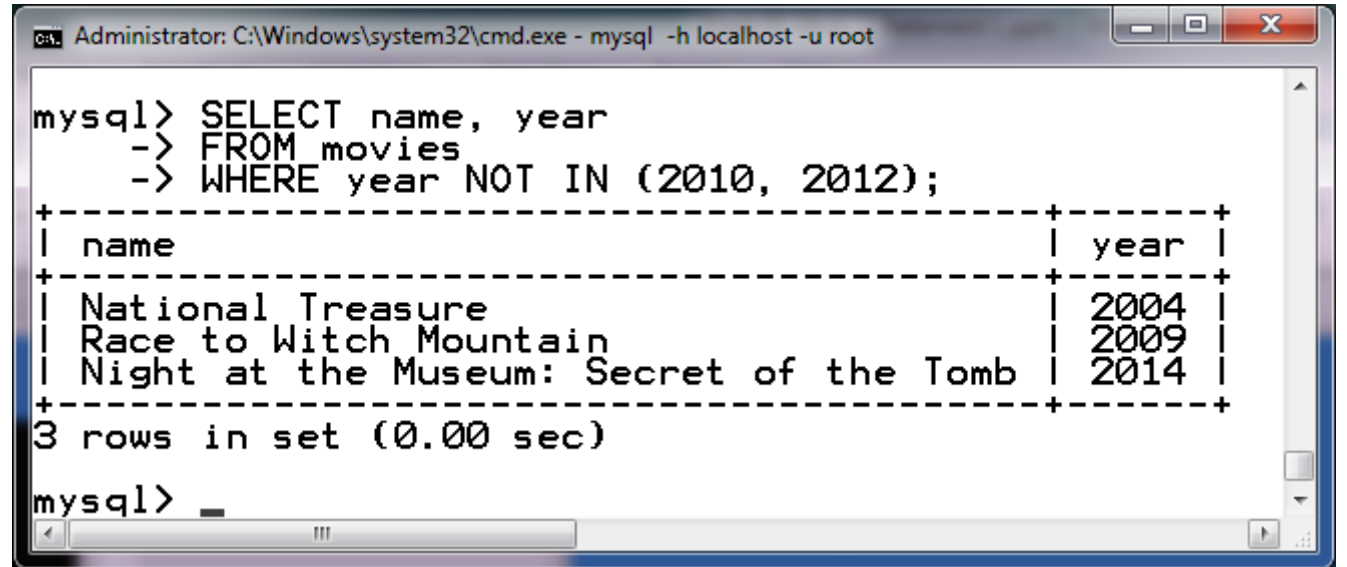
name	year
The Sorcerer's Apprentice	2010
The Hunger Games	2012
Tooth Fairy	2010
Journey 2: The Mysterious Island	2012

4 rows in set (0.00 sec)

mysql> _

IN OPERATOR

```
SELECT name, year  
FROM movies  
WHERE year NOT IN (2010, 2012);
```

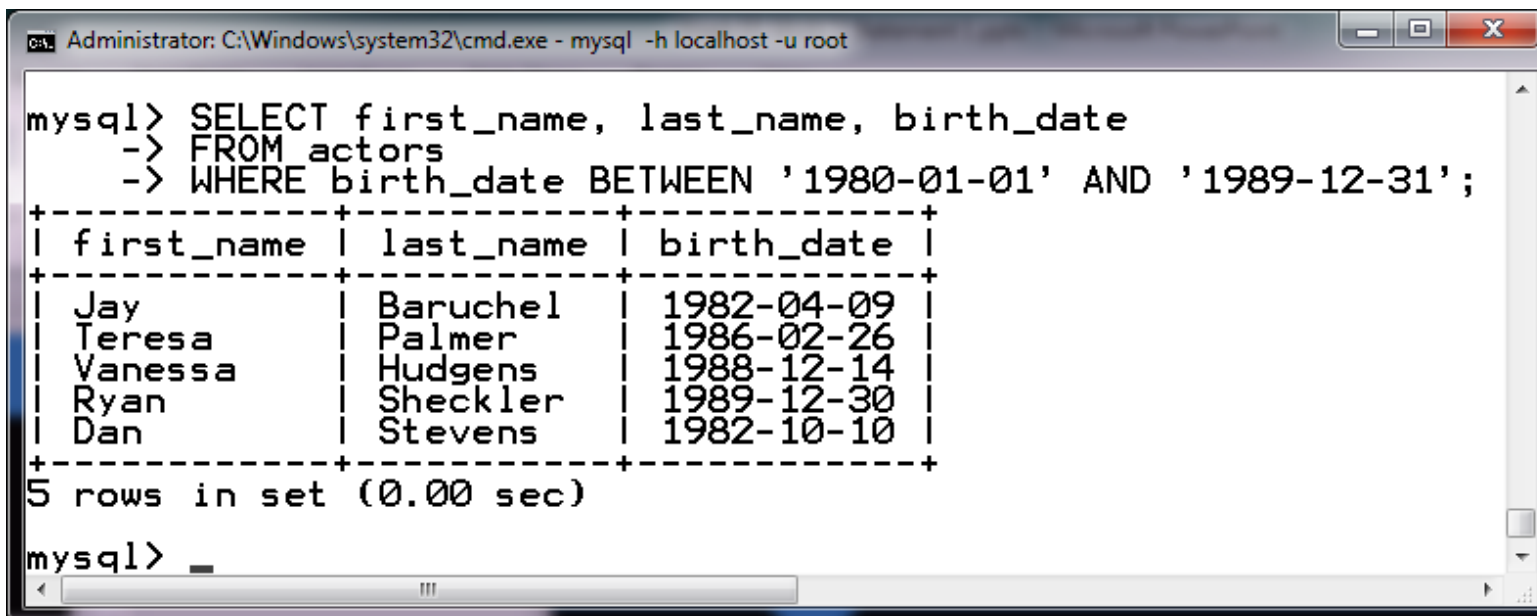


The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". The prompt shows the execution of a MySQL query: `mysql> SELECT name, year -> FROM movies -> WHERE year NOT IN (2010, 2012);`. The output is a table with two columns: "name" and "year". The table contains three rows of data: "National Treasure" (2004), "Race to Witch Mountain" (2009), and "Night at the Museum: Secret of the Tomb" (2014). Below the table, it says "3 rows in set (0.00 sec)". The prompt ends with `mysql> _`.

name	year
National Treasure	2004
Race to Witch Mountain	2009
Night at the Museum: Secret of the Tomb	2014

BETWEEN OPERATOR

```
SELECT first_name, last_name, birth_date  
FROM actors  
WHERE birth_date BETWEEN '1980-01-01' AND '1989-12-31';
```



The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". The prompt is "mysql>". The user enters the following SQL query:

```
mysql> SELECT first_name, last_name, birth_date  
-> FROM actors  
-> WHERE birth_date BETWEEN '1980-01-01' AND '1989-12-31';
```

The output is a table with 5 rows and 3 columns: first_name, last_name, and birth_date. The data is as follows:

first_name	last_name	birth_date
Jay	Baruchel	1982-04-09
Teresa	Palmer	1986-02-26
Vanessa	Hudgens	1988-12-14
Ryan	Sheckler	1989-12-30
Dan	Stevens	1982-10-10

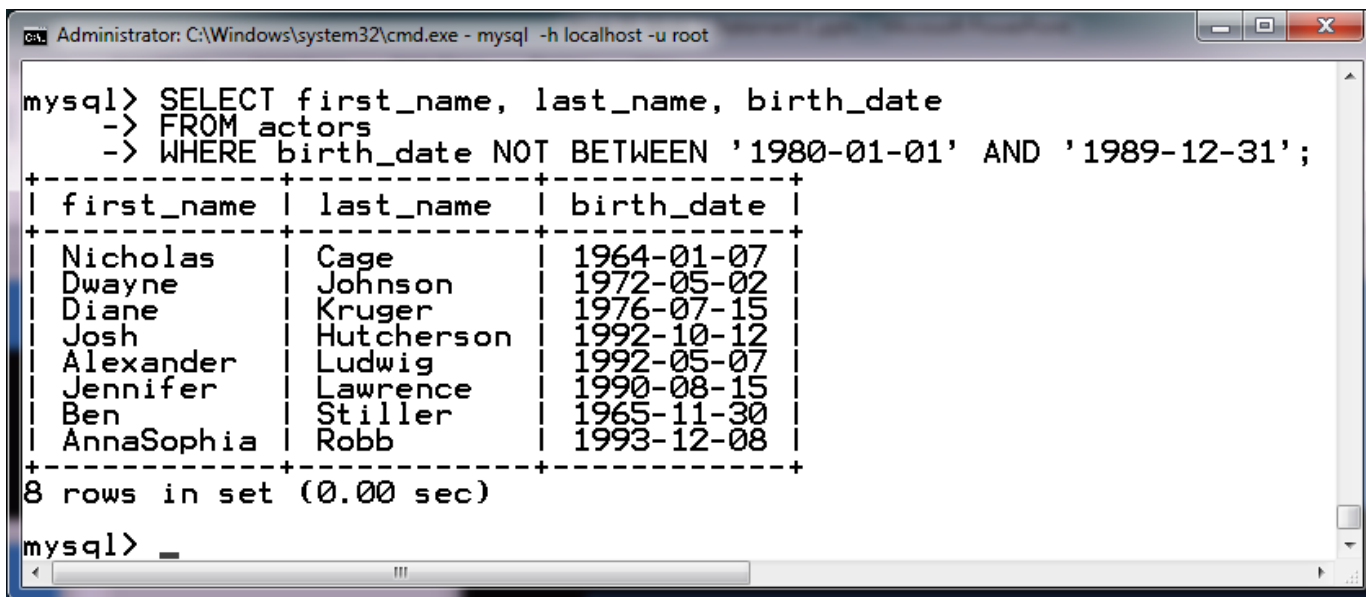
Below the table, the prompt shows "5 rows in set (0.00 sec)". The prompt is now "mysql> _".

BETWEEN OPERATOR

SELECT first_name, last_name, birth_date

FROM actors

WHERE birth_date NOT BETWEEN '1980-01-01' AND '1989-12-31';



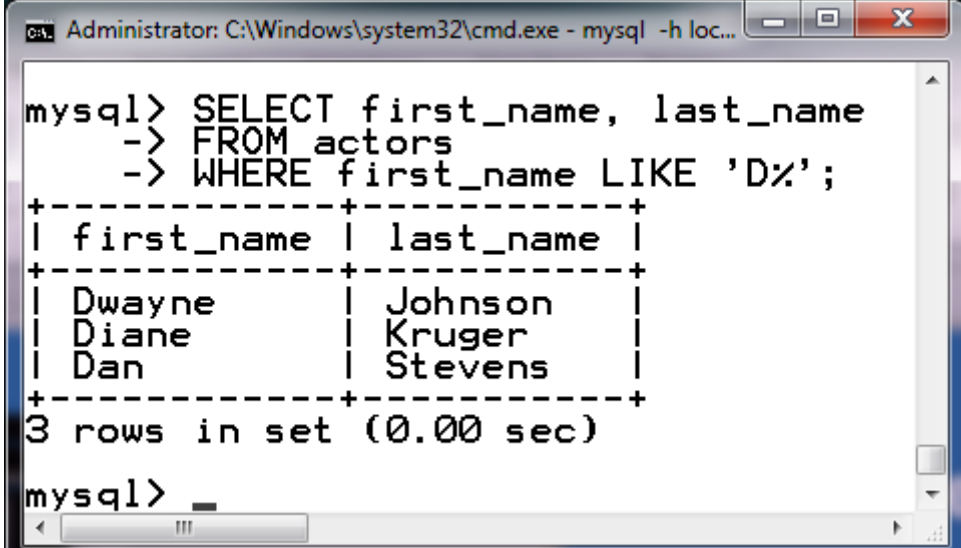
The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". Inside the window, a MySQL prompt "mysql>" is followed by the SQL query: "SELECT first_name, last_name, birth_date FROM actors WHERE birth_date NOT BETWEEN '1980-01-01' AND '1989-12-31';". The query is executed, and the results are displayed in a table format with columns "first_name", "last_name", and "birth_date". There are 8 rows of data. Below the table, it says "8 rows in set (0.00 sec)". The prompt "mysql>" is shown again at the bottom.

```
mysql> SELECT first_name, last_name, birth_date
-> FROM actors
-> WHERE birth_date NOT BETWEEN '1980-01-01' AND '1989-12-31';
+-----+-----+-----+
| first_name | last_name | birth_date |
+-----+-----+-----+
| Nicholas  | Cage      | 1964-01-07 |
| Dwayne    | Johnson  | 1972-05-02 |
| Diane     | Kruger   | 1976-07-15 |
| Josh      | Hutcherson | 1992-10-12 |
| Alexander | Ludwig   | 1992-05-07 |
| Jennifer  | Lawrence  | 1990-08-15 |
| Ben       | Stiller   | 1965-11-30 |
| AnnaSophia | Robb     | 1993-12-08 |
+-----+-----+-----+
8 rows in set (0.00 sec)

mysql>
```

LIKE OPERATOR

```
SELECT first_name, last_name  
FROM actors  
WHERE first_name LIKE 'D%';
```



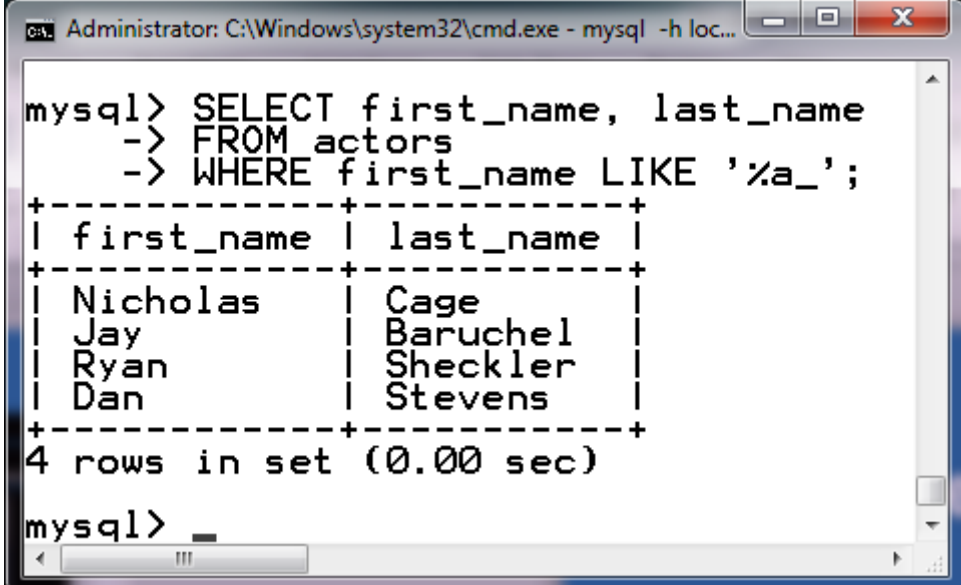
A screenshot of a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h loc...". The window shows a MySQL session with the following text:

```
mysql> SELECT first_name, last_name  
-> FROM actors  
-> WHERE first_name LIKE 'D%';  
+-----+-----+  
| first_name | last_name |  
+-----+-----+  
| Dwayne    | Johnson  |  
| Diane     | Kruger   |  
| Dan       | Stevens  |  
+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql> _
```

first_name	last_name
Dwayne	Johnson
Diane	Kruger
Dan	Stevens

LIKE OPERATOR

```
SELECT first_name, last_name  
FROM actors  
WHERE first_name LIKE '%a_';
```

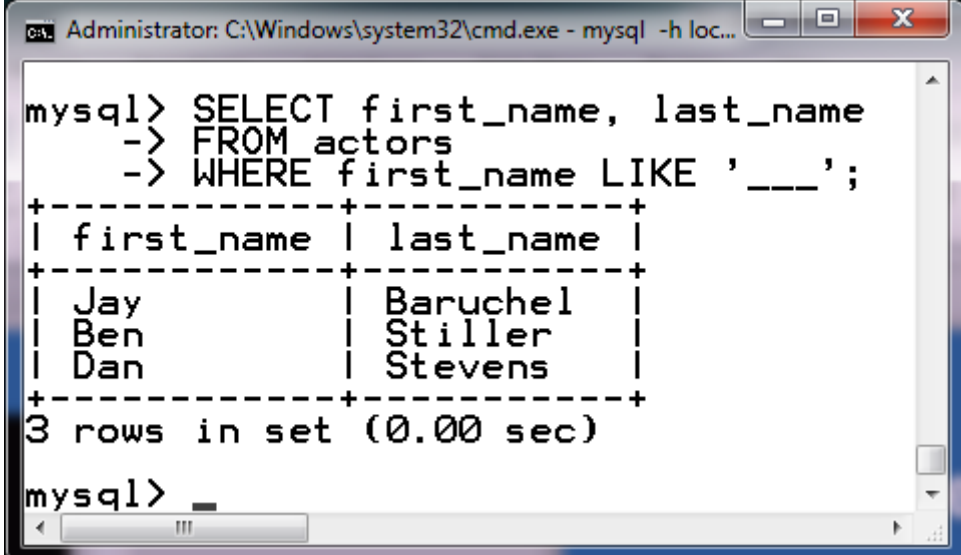


The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h loc...". Inside the window, a MySQL query is executed, and the results are displayed in a table format.

```
mysql> SELECT first_name, last_name  
-> FROM actors  
-> WHERE first_name LIKE '%a_';  
+-----+-----+  
| first_name | last_name |  
+-----+-----+  
| Nicholas  | Cage      |  
| Jay       | Baruchel  |  
| Ryan      | Sheckler  |  
| Dan       | Stevens   |  
+-----+-----+  
4 rows in set (0.00 sec)  
  
mysql> _
```

LIKE OPERATOR

```
SELECT first_name, last_name  
FROM actors  
WHERE first_name LIKE '___';
```

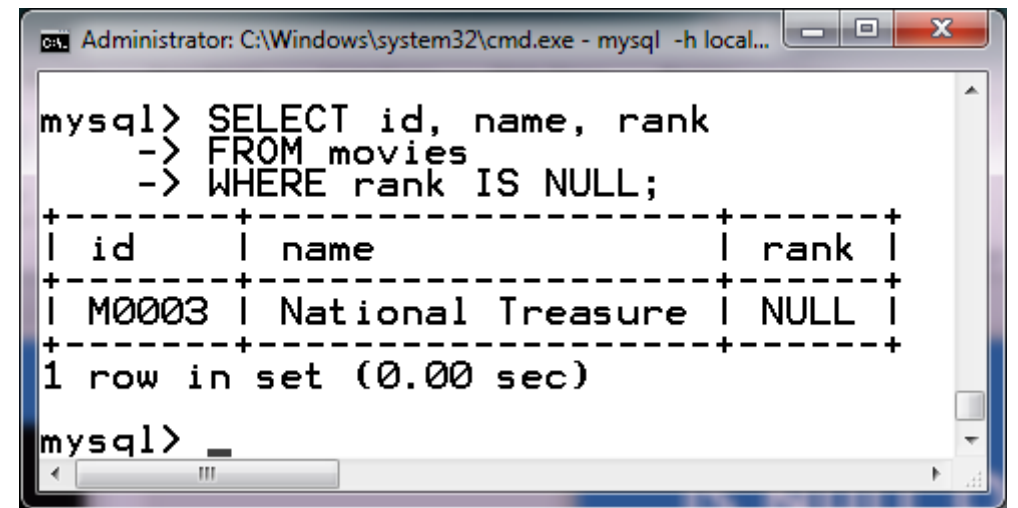


The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h loc...". Inside the window, a MySQL query is executed, and the results are displayed in a table format. The query is: `mysql> SELECT first_name, last_name -> FROM actors -> WHERE first_name LIKE '___';`. The results show three rows: Jay Baruchel, Ben Stiller, and Dan Stevens. Below the table, it says "3 rows in set (0.00 sec)". The prompt `mysql>` is visible at the bottom.

```
mysql> SELECT first_name, last_name  
-> FROM actors  
-> WHERE first_name LIKE '___';  
+-----+-----+  
| first_name | last_name |  
+-----+-----+  
| Jay        | Baruchel  |  
| Ben        | Stiller   |  
| Dan        | Stevens   |  
+-----+-----+  
3 rows in set (0.00 sec)  
mysql>
```

IS NULL OPERATOR

```
SELECT id, name, rank
FROM movies
WHERE rank IS NULL;
```



The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h local...". Inside the window, the following MySQL query is entered and executed:

```
mysql> SELECT id, name, rank
-> FROM movies
-> WHERE rank IS NULL;
```

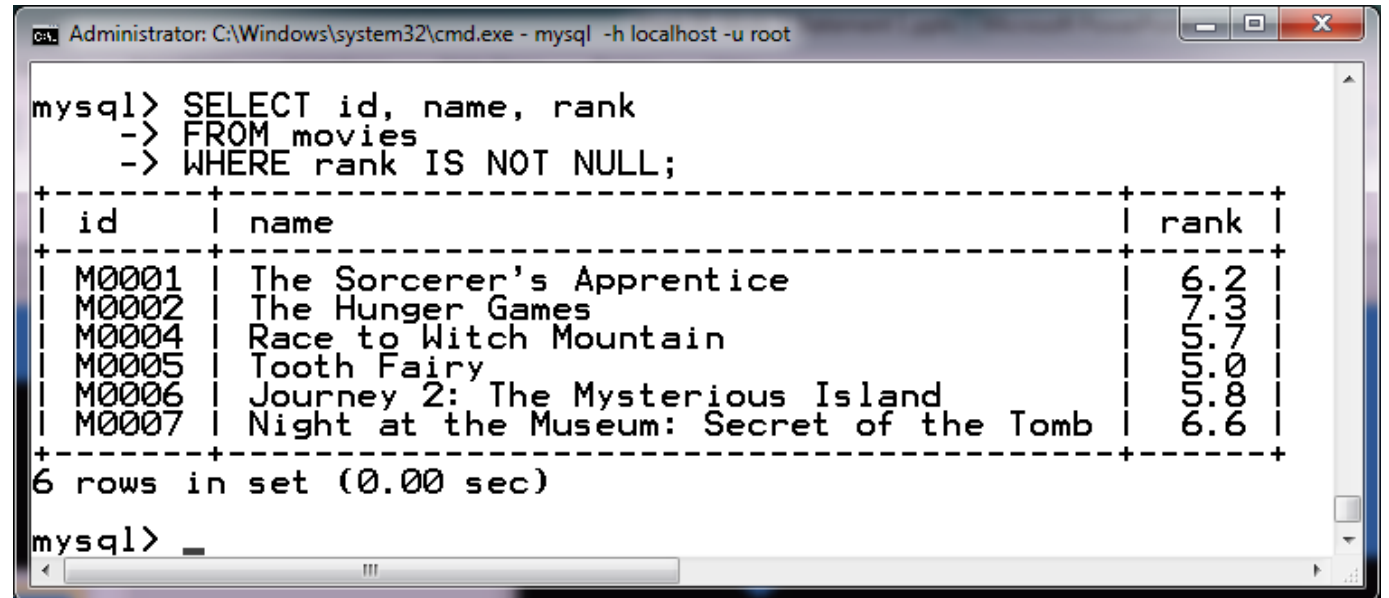
The output is displayed in a table format with dashed lines as separators:

id	name	rank
M0003	National Treasure	NULL

Below the table, the text "1 row in set (0.00 sec)" is shown. The prompt "mysql>" is visible at the bottom of the window.

IS NULL OPERATOR

```
SELECT id, name, rank  
FROM movies  
WHERE rank IS NOT NULL;
```



The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". The prompt is "mysql>". The user has entered the following SQL query:

```
mysql> SELECT id, name, rank  
-> FROM movies  
-> WHERE rank IS NOT NULL;
```

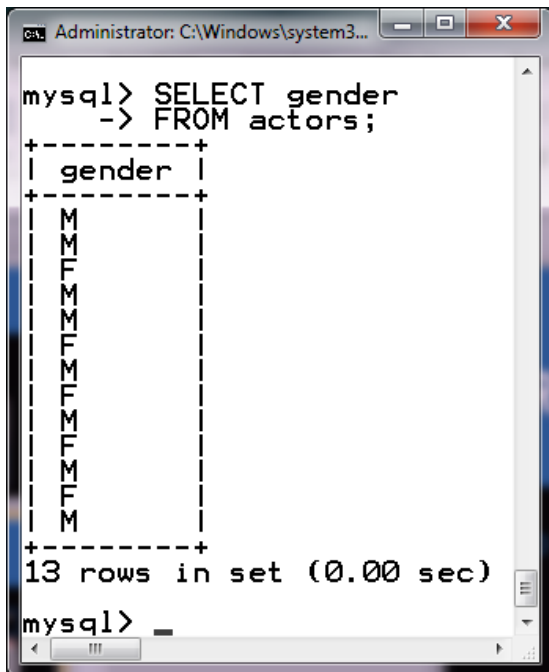
The output is a table with 6 rows and 3 columns: id, name, and rank. The data is as follows:

id	name	rank
M0001	The Sorcerer's Apprentice	6.2
M0002	The Hunger Games	7.3
M0004	Race to Witch Mountain	5.7
M0005	Tooth Fairy	5.0
M0006	Journey 2: The Mysterious Island	5.8
M0007	Night at the Museum: Secret of the Tomb	6.6

Below the table, the prompt shows "6 rows in set (0.00 sec)". The prompt is now "mysql> _".

REMOVING DUPLICATE ROWS: DISTINCT

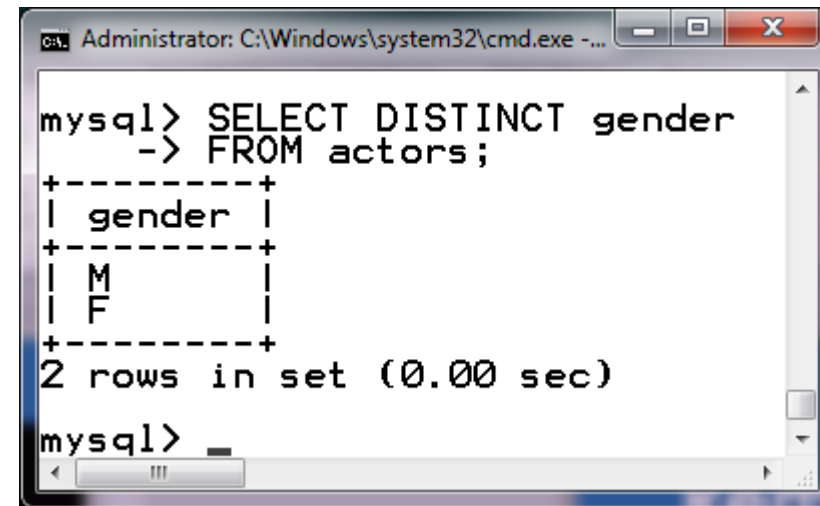
SELECT gender
FROM actors;



A screenshot of a MySQL command window titled "Administrator: C:\Windows\system32\cmd.exe". The prompt is "mysql>". The user enters the command "SELECT gender" followed by a new line and "-> FROM actors;". The output is a table with a single column "gender" containing 13 rows of data. The first row is "M", followed by 12 rows of "F". The output is enclosed in a dashed box. Below the table, it says "13 rows in set (0.00 sec)". The prompt "mysql> _" is visible at the bottom.

```
mysql> SELECT gender
-> FROM actors;
+-----+
| gender |
+-----+
| M      |
| F      |
| F      |
| F      |
| F      |
| F      |
| F      |
| F      |
| F      |
| F      |
| F      |
| F      |
| F      |
+-----+
13 rows in set (0.00 sec)
mysql> _
```

SELECT DISTINCT gender
FROM actors;



A screenshot of a MySQL command window titled "Administrator: C:\Windows\system32\cmd.exe". The prompt is "mysql>". The user enters the command "SELECT DISTINCT gender" followed by a new line and "-> FROM actors;". The output is a table with a single column "gender" containing 2 rows of data: "M" and "F". The output is enclosed in a dashed box. Below the table, it says "2 rows in set (0.00 sec)". The prompt "mysql> _" is visible at the bottom.

```
mysql> SELECT DISTINCT gender
-> FROM actors;
+-----+
| gender |
+-----+
| M      |
| F      |
+-----+
2 rows in set (0.00 sec)
mysql> _
```

AGGREGATION FUNCTIONS: COUNT FUNCTION

```
SELECT COUNT(*)  
FROM actors;
```



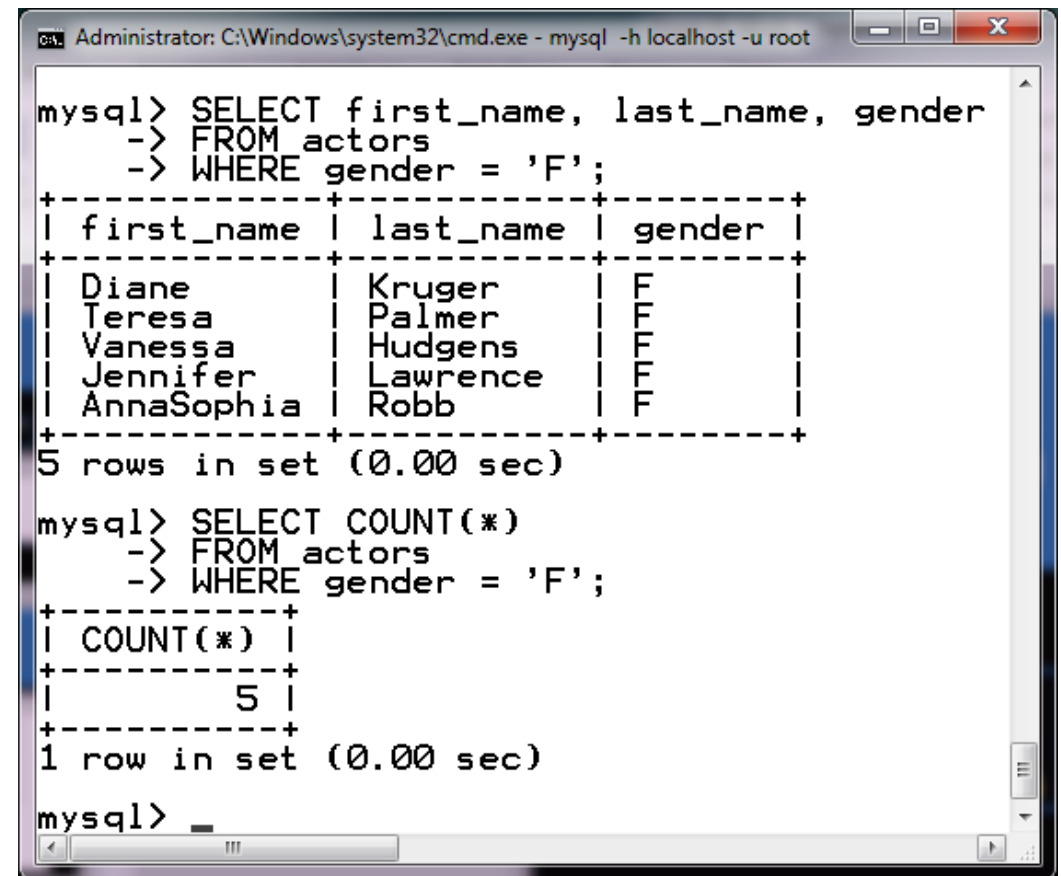
```
Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u ...  
  
mysql> SELECT id, first_name, last_name  
-> FROM actors;  
  
+-----+-----+-----+  
| id      | first_name | last_name |  
+-----+-----+-----+  
| A0001   | Nicholas   | Cage      |  
| A0002   | Dwayne     | Johnson   |  
| A0003   | Diane      | Kruger    |  
| A0004   | Jay        | Baruchel  |  
| A0005   | Josh       | Hutcherson|  
| A0006   | Teresa     | Palmer    |  
| A0007   | Alexander  | Ludwig    |  
| A0008   | Vanessa    | Hudgens   |  
| A0009   | Ryan       | Sheckler  |  
| A0010   | Jennifer   | Lawrence  |  
| A0011   | Ben        | Stiller   |  
| A0012   | AnnaSophia| Robb      |  
| A0013   | Dan        | Stevens   |  
+-----+-----+-----+  
13 rows in set (0.00 sec)  
  
mysql> SELECT COUNT(*)  
-> FROM actors;  
  
+-----+  
| COUNT(*) |  
+-----+  
| 13       |  
+-----+  
1 row in set (0.00 sec)  
  
mysql>
```

AGGREGATION FUNCTIONS: COUNT FUNCTION

SELECT COUNT(*)

FROM actors

WHERE gender = 'F';



```
Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root

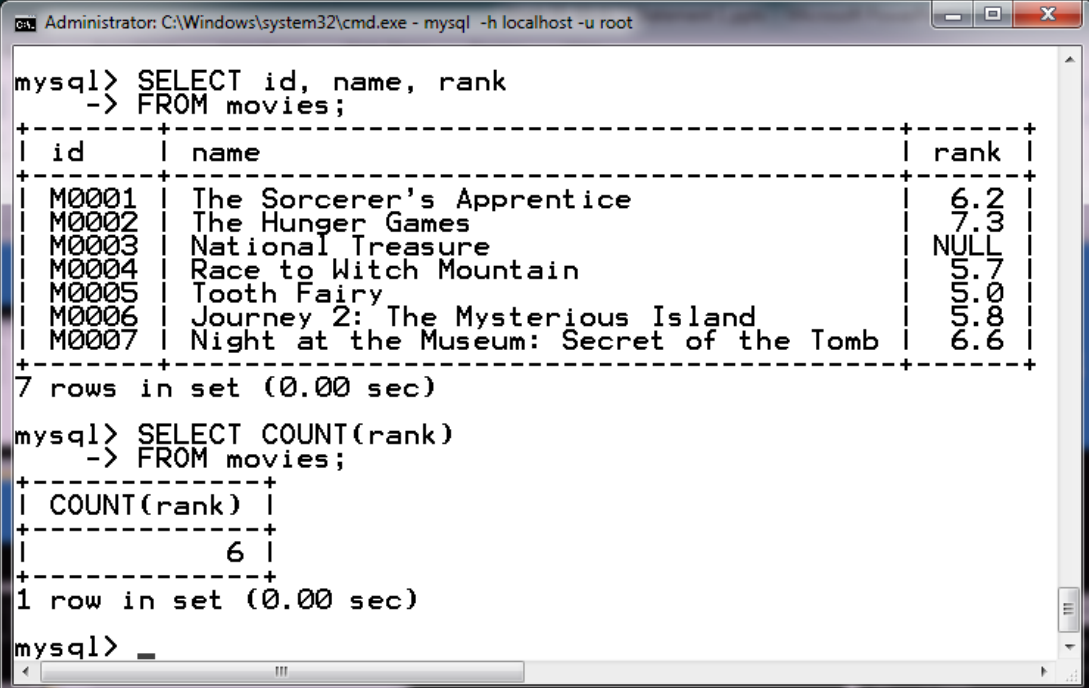
mysql> SELECT first_name, last_name, gender
-> FROM actors
-> WHERE gender = 'F';
+-----+-----+-----+
| first_name | last_name | gender |
+-----+-----+-----+
| Diane      | Kruger    | F      |
| Teresa     | Palmer    | F      |
| Vanessa    | Hudgens   | F      |
| Jennifer   | Lawrence  | F      |
| AnnaSophia | Robb      | F      |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT COUNT(*)
-> FROM actors
-> WHERE gender = 'F';
+-----+
| COUNT(*) |
+-----+
|         5 |
+-----+
1 row in set (0.00 sec)

mysql> _
```

AGGREGATION FUNCTIONS: COUNT FUNCTION

```
SELECT COUNT(rank)
FROM movies;
```



```
Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root

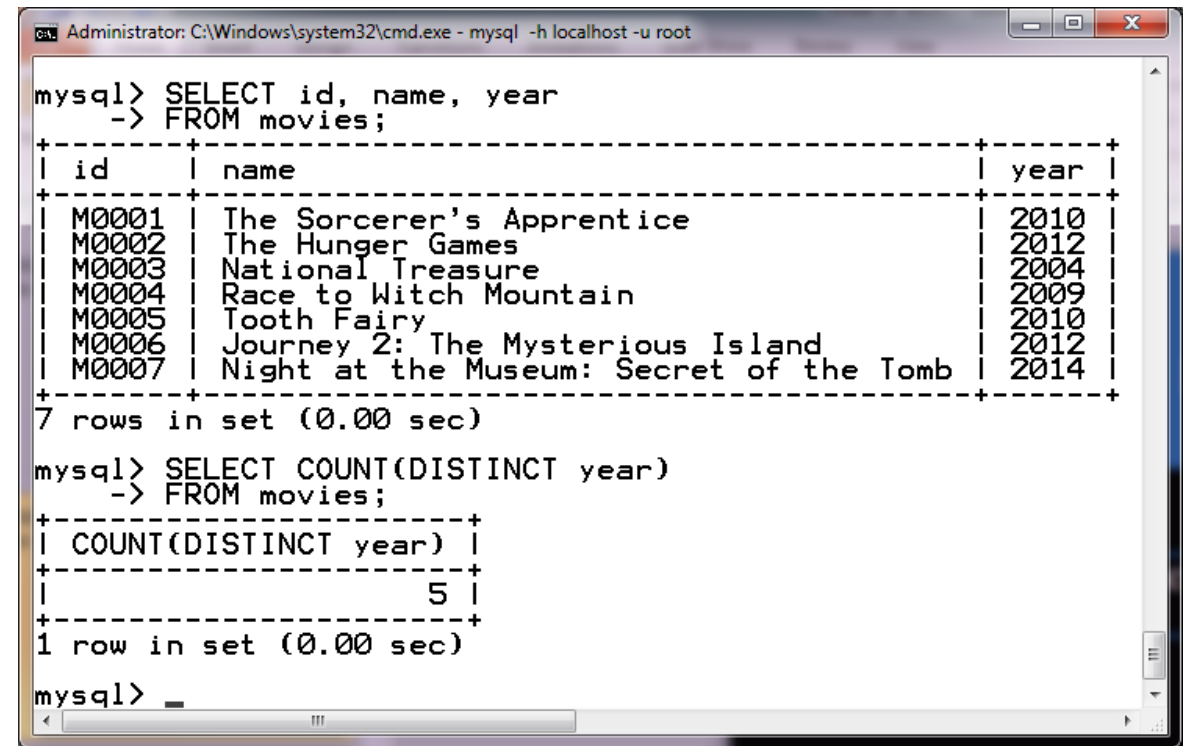
mysql> SELECT id, name, rank
-> FROM movies;
+-----+-----+-----+
| id    | name                                     | rank |
+-----+-----+-----+
| M0001 | The Sorcerer's Apprentice              | 6.2  |
| M0002 | The Hunger Games                       | 7.3  |
| M0003 | National Treasure                     | NULL |
| M0004 | Race to Witch Mountain                | 5.7  |
| M0005 | Tooth Fairy                           | 5.0  |
| M0006 | Journey 2: The Mysterious Island       | 5.8  |
| M0007 | Night at the Museum: Secret of the Tomb | 6.6  |
+-----+-----+-----+
7 rows in set (0.00 sec)

mysql> SELECT COUNT(rank)
-> FROM movies;
+-----+
| COUNT(rank) |
+-----+
| 6           |
+-----+
1 row in set (0.00 sec)

mysql> _
```

AGGREGATION FUNCTIONS: COUNT FUNCTION

```
SELECT COUNT(DISTINCT year)
FROM movies;
```



The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". It displays two SQL queries and their results in a MySQL shell.

```
mysql> SELECT id, name, year
-> FROM movies;
```

id	name	year
M0001	The Sorcerer's Apprentice	2010
M0002	The Hunger Games	2012
M0003	National Treasure	2004
M0004	Race to Witch Mountain	2009
M0005	Tooth Fairy	2010
M0006	Journey 2: The Mysterious Island	2012
M0007	Night at the Museum: Secret of the Tomb	2014

```
7 rows in set (0.00 sec)
```

```
mysql> SELECT COUNT(DISTINCT year)
-> FROM movies;
```

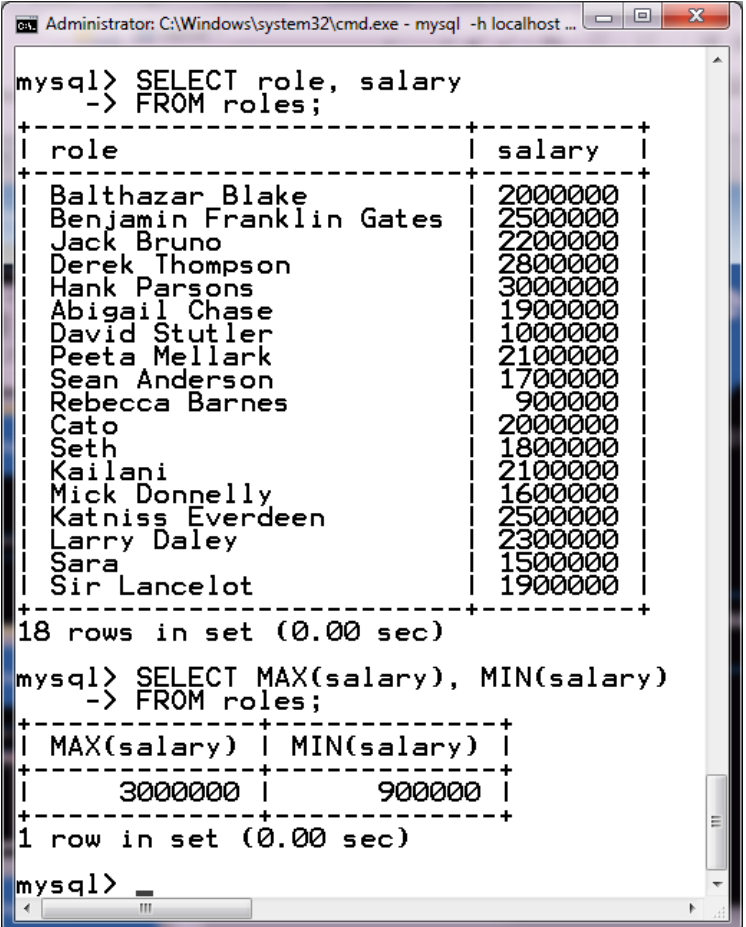
COUNT(DISTINCT year)
5

```
1 row in set (0.00 sec)
```

```
mysql>
```

AGGREGATION FUNCTIONS: MAX & MIN FUNCTIONS

```
SELECT MAX(salary), MIN(salary)
FROM roles;
```



```
Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost ...
mysql> SELECT role, salary
-> FROM roles;
+-----+-----+
| role                                | salary |
+-----+-----+
| Balthazar Blake                     | 2000000 |
| Benjamin Franklin Gates             | 2500000 |
| Jack Bruno                          | 2200000 |
| Derek Thompson                      | 2800000 |
| Hank Parsons                       | 3000000 |
| Abigail Chase                      | 1900000 |
| David Stutler                       | 1000000 |
| Peeta Mellark                       | 2100000 |
| Sean Anderson                      | 1700000 |
| Rebecca Barnes                     | 900000  |
| Cato                                | 2000000 |
| Seth                                | 1800000 |
| Kailani                             | 2100000 |
| Mick Donnelly                       | 1600000 |
| Katniss Everdeen                   | 2500000 |
| Larry Daley                        | 2300000 |
| Sara                               | 1500000 |
| Sir Lancelot                       | 1900000 |
+-----+-----+
18 rows in set (0.00 sec)

mysql> SELECT MAX(salary), MIN(salary)
-> FROM roles;
+-----+-----+
| MAX(salary) | MIN(salary) |
+-----+-----+
| 3000000     | 900000      |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

AGGREGATION FUNCTIONS: MAX & MIN FUNCTIONS

```
SELECT MAX(first_name),  
MIN(first_name) FROM actors;
```

```
Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root  
mysql> SELECT first_name, last_name, birth_date  
-> FROM actors;  
+-----+-----+-----+  
| first_name | last_name | birth_date |  
+-----+-----+-----+  
| Nicholas  | Cage      | 1964-01-07 |  
| Dwayne    | Johnson   | 1972-05-02 |  
| Diane     | Kruger    | 1976-07-15 |  
| Jay       | Baruchel  | 1982-04-09 |  
| Josh      | Hutcherson | 1992-10-12 |  
| Teresa    | Palmer     | 1986-02-26 |  
| Alexander | Ludwig    | 1992-05-07 |  
| Vanessa   | Hudgens   | 1988-12-14 |  
| Ryan      | Sheckler  | 1989-12-30 |  
| Jennifer  | Lawrence  | 1990-08-15 |  
| Ben       | Stiller   | 1965-11-30 |  
| AnnaSophia | Robb      | 1993-12-08 |  
| Dan       | Stevens   | 1982-10-10 |  
+-----+-----+-----+  
13 rows in set (0.00 sec)  
mysql>
```

```
Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root  
mysql> SELECT MAX(first_name), MIN(first_name)  
-> FROM actors;  
+-----+-----+  
| MAX(first_name) | MIN(first_name) |  
+-----+-----+  
| Vanessa          | Alexander        |  
+-----+-----+  
1 row in set (0.00 sec)  
mysql>
```

AGGREGATION FUNCTIONS: MAX & MIN FUNCTIONS

```
SELECT MAX(birth_date),  
MIN(birth_date) FROM actors;
```

```
Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root  
mysql> SELECT first_name, last_name, birth_date  
-> FROM actors;  
+-----+-----+-----+  
| first_name | last_name | birth_date |  
+-----+-----+-----+  
| Nicholas  | Cage      | 1964-01-07 |  
| Dwayne    | Johnson  | 1972-05-02 |  
| Diane     | Kruger    | 1976-07-15 |  
| Jay       | Baruchel  | 1982-04-09 |  
| Josh      | Hutcherson | 1992-10-12 |  
| Teresa    | Palmer    | 1986-02-26 |  
| Alexander | Ludwig    | 1992-05-07 |  
| Vanessa   | Hudgens   | 1988-12-14 |  
| Ryan      | Sheckler  | 1989-12-30 |  
| Jennifer  | Lawrence  | 1990-08-15 |  
| Ben       | Stiller   | 1965-11-30 |  
| AnnaSophia | Robb     | 1993-12-08 |  
| Dan       | Stevens   | 1982-10-10 |  
+-----+-----+-----+  
13 rows in set (0.00 sec)  
mysql>
```

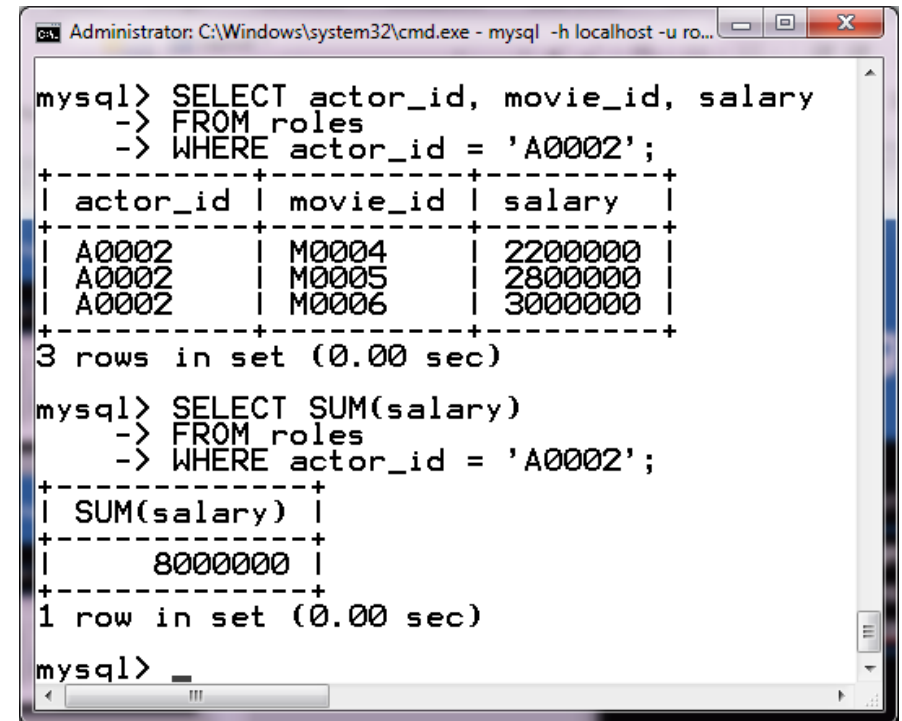
```
Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root  
mysql> SELECT MAX(birth_date), MIN(birth_date)  
-> FROM actors;  
+-----+-----+  
| MAX(birth_date) | MIN(birth_date) |  
+-----+-----+  
| 1993-12-08      | 1964-01-07      |  
+-----+-----+  
1 row in set (0.00 sec)  
mysql>
```

AGGREGATION FUNCTIONS: SUM FUNCTION

SELECT SUM(salary)

FROM roles

WHERE actor_id = 'A0002';



```
Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u ro...

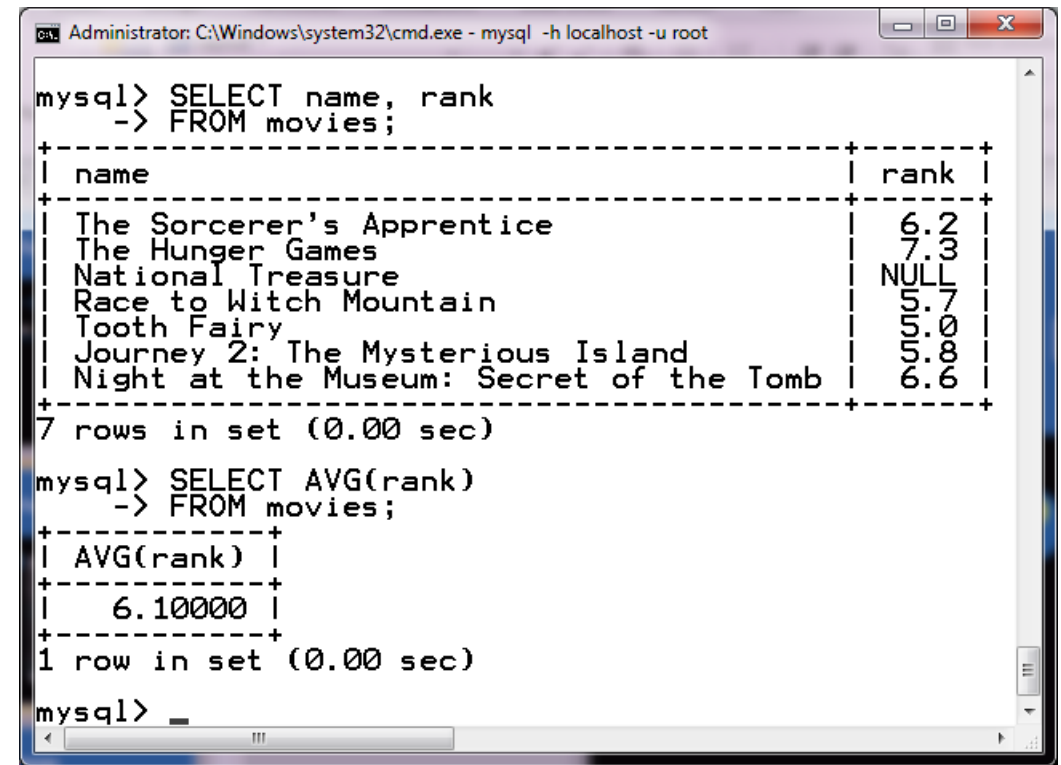
mysql> SELECT actor_id, movie_id, salary
-> FROM roles
-> WHERE actor_id = 'A0002';
+-----+-----+-----+
| actor_id | movie_id | salary |
+-----+-----+-----+
| A0002    | M0004    | 2200000 |
| A0002    | M0005    | 2800000 |
| A0002    | M0006    | 3000000 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT SUM(salary)
-> FROM roles
-> WHERE actor_id = 'A0002';
+-----+
| SUM(salary) |
+-----+
| 8000000     |
+-----+
1 row in set (0.00 sec)

mysql>
```

AGGREGATION FUNCTIONS:AVG FUNCTION

```
SELECT AVG(rank)
FROM movies;
```



The screenshot shows a Windows command prompt window titled "Administrator: C:\Windows\system32\cmd.exe - mysql -h localhost -u root". It displays two SQL queries and their results in a terminal-like format.

```
mysql> SELECT name, rank
-> FROM movies;
```

name	rank
The Sorcerer's Apprentice	6.2
The Hunger Games	7.3
National Treasure	NULL
Race to Witch Mountain	5.7
Tooth Fairy	5.0
Journey 2: The Mysterious Island	5.8
Night at the Museum: Secret of the Tomb	6.6

```
7 rows in set (0.00 sec)

mysql> SELECT AVG(rank)
-> FROM movies;
```

AVG(rank)
6.10000

```
1 row in set (0.00 sec)

mysql> _
```

REFERENCES

- Elmasri, Ramez and Shamkant B. Navathe (2016), Fundamentals of Database Systems, 7th edition, Addison Wesley.
- Rick F.van der Lans (2007), SQL for MySQL Developers, Addison Wesley.
- Connolly, Thomas and Carolyn Begg (2015), Database Systems A Practical Approach to Design, Implementation, and Management, 6th edition, Pearson Education, ISBN 13: 978-1-292-06118-4, Essex, England.

NEXT WEEK: SELECT STATEMENT 2

- GROUP BY
- HAVING
- ORDER BY
- LIMIT
- CASE Expression
- Subquery

```
SELECT *  
FROM WORLD  
WHERE SOMEONE  
LIKE '%YOU%';
```

```
EMPTY SET (0.00 SEC)
```

Vision

To become an **outstanding** undergraduate Computer Science program that produces **international-minded** graduates who are **competent** in software engineering and have **entrepreneurial spirit** and **noble character**.



Mission

1. To conduct studies with the best technology and curriculum, supported by professional lecturer
2. To conduct research in Informatics to promote science and technology
3. To deliver science-and-technology-based society services to implement science and technology